RP-01 — Application Inventory & Baseline Extraction: Group Lesson Planner

Metadata

- **Report id:** RP-01
- **Date:** 2026-08-20
- **Subject:** Group Lesson Planner — `https://denyslystopadskyy.github.io/lesson-planner/`, single static file, no backend
- **Source of truth for line citations:** `index.html` in the research worktree at commit `211eb6b` (1473 lines, 58,649 bytes), confirmed byte-identical to the deployed artifact
- **Inputs consumed:** the orchestrator's static baseline notes; `index.html` read in full; `AGENTS.md`; the live deployment over HTTPS; uncommitted local prior art in the main checkout (`docs/functionality.md`, `docs/tech-details.md`, `docs/bdd-usage.md`, `docs/qa-coverage-investigation.md`, `e2e/**`, `playwright.config.ts`, `package.json`, `tsconfig.json`, and a **modified working copy of `index.html`**)
- **Verification statement:** Feature behaviour, storage shape, persistence, clipboard, CSV round-trip, dialog strings, tab order, colour values, viewport behaviour and 24 of the 26 defects in section 8 (all but D20 and D25) were **runtime-verified** in a real browser against a locally served byte-identical copy of the deployed file; code structure, line numbers, dead code and string literals were **statically read** from `index.html`; anything reasoned rather than observed is labelled **inference** inline.

Executive summary

The app is a single 58,649-byte HTML file with zero dependencies, zero build tooling and zero network activity after load. It is a **monthly lesson-schedule and fee calculator for teaching groups**, ending in a copy-to-clipboard payment message. There is no student entity, no attendance and no payment tracking — the research pack's framing of "messages to her students" describes intent, not the data model.

Four findings dominate everything downstream:

1. **A real person's full name, bank IBAN and tax identification number are hardcoded in the default payment template** (`index.html:387–392`, with a personal first name at `index.html:400`). They are publicly readable via view-source on the live site — runtime-verified, since the values reach the generated message for any user who never edits the template. Per the orchestrator's git read, which this agent did not independently verify, they have been present since the app's first commit, so removing them from `HEAD` would not remove them from history. This is a live personal-data exposure, not a code-quality nit.
2. **Storage has no error handling of any kind, and the resulting failure is a silently dead page.** `JSON.parse` runs unguarded inside `App.init()` (`index.html:1188–1198`). Runtime-verified by planting a truncated JSON value in `groupLessonPlannerData` and reloading: an uncaught `SyntaxError` aborts `init()` after `cacheElements()` but before the month dropdown is populated,

so the group list renders **nothing at all — not even the empty state**, the month `<select>` has zero options, `bindEvents()` never runs, and clicking `+ Add Group` does nothing. The user sees a title and dead buttons, with no error and no route to recovery. This is the most plausible mechanism for a future repeat of the data loss that motivated this programme.

3. **The app can create data that permanently breaks its own only backup path.** Typing a 1-digit year into the unconstrained year input persists malformed keys such as `5-08-10`, and re-importing the app's own CSV export then throws `Invalid month format`, aborting the entire restore. Runtime-verified end to end.
4. **The core workflow is unusable by keyboard.** Group cards and all 31 day cells plus 7 weekday headers are non-focusable `div`s. Meanwhile 9 of the 14 tab stops on a loaded page belong to *closed, invisible* modals, and activating them causes an uncaught page error, a stray write to `paymentTemplate`, and a silent clipboard overwrite.

Beyond the orchestrator's list I found five defects it did not have, of which two are data-loss class: committing a price change **silently reverts an unsaved group-name edit**, and a group whose currency is not exactly three letters becomes **permanently unopenable**. I also found that the developer's own uncommitted working copy already fixes the closed-modal problem and adds test hooks — valuable prior art, but it means the prior-art docs' line numbers point at a 1491-line file that is not what is deployed.

1. Access report

Target	Method	Result
Live app	HTTPS GET plus browser load	HTTP 200, 58,649 bytes uncompressed, 13,610 bytes gzipped, `last-modified: Mon, 13 Oct 2025 14:16:26 GMT`, `etag: "68ed09ba-e519"` (`e519` hex = 58,649)
Deployed vs committed source	Byte diff of fetched HTML against worktree `index.html`	**Identical.** Every `index.html:LINE` citation in this report therefore describes the artifact real users load
Interactive session	Playwright against `http://127.0.0.1:4173/index.html` serving the same bytes	Chosen over the live origin so writes landed on a throwaway origin, and over `file://` because `navigator.clipboard` requires a secure context; `window.isSecureContext === true` was confirmed
Repo, worktree	Direct read	Tracked: `index.html`, `AGENTS.md`, `LICENSE`, `.gitignore`. No README, no `package.json`, no `.github/`, no CI config
Prior art, main checkout	Direct read	Uncommitted `docs/*.md` (4 files), `e2e/**` (38 files), `playwright.config.ts`, `package.json`, `tsconfig.json`, **and a modified `index.html`**
GitHub Pages configuration	Not reachable	Repository Settings are not exposed to an unauthenticated client. Branch-based publishing is the only remaining explanation given no workflow file exists [S7]

Target	Method	Result
Git history	Not run	This agent was instructed not to run git commands. History claims below are attributed to the orchestrator's read, not independently verified
The end user's real dataset	Gone	Lost with the laptop wipe. All volume figures come from data I created

Two divergent copies of index.html

This matters more than it looks, and every downstream report needs it.

Copy	Lines	Bytes	Status
Worktree / committed / **deployed**	1473	58,649	What users run today
Main checkout working copy	1491	59,604	**Uncommitted**, never deployed

The uncommitted copy is not noise — it is the developer's own direction of travel, and it already fixes one of this report's most serious accessibility defects:

- Adds `hidden` to all three modal overlays plus a `[hidden] { display:none !important; }` rule, and toggles `.hidden` in every open/close handler. This removes closed modals from the accessibility tree and the tab order.
- Adds `data-testid` and `data-*` hooks: `data-group-name`, `data-group-index`, `data-testid="group-card-name"`, `group-card-lesson-count`, `data-month-key`, `month-name`, `month-lesson-count`, `month-total`, `month-price-input`, `price-per-lesson`, `copy-payment-message`, `data-weekday`, `data-date`, `data-day`.

Consequence for the programme: the deployed file contains **zero** `data-testid` and **zero** `data-*` attributes — runtime-verified. Section 7 lists anchors present in the deployed file only. Separately, `docs/qa-coverage-investigation.md` cites line numbers (for example `index.html:1195-1218` for the storage service, which is actually at 1187-1213 in the deployed file) that resolve against the 1491-line working copy. Any later report that follows those citations lands on the wrong lines.

TBD items

Item	Value	Why unresolved	What resolves it
localStorage quota for this origin	`TBD`	Not measured; browser- and platform-dependent [S2]	Run `navigator.storage.estimate()` in the target browser, or probe by writing until `QuotaExceededError`
GitHub Pages publishing source, branch and folder	`TBD`	Repository Settings are not publicly readable	Open Settings then Pages on `https://github.com/denyslystopadskyy/lesson-planner`

Item	Value	Why unresolved	What resolves it
Whether the uncommitted `index.html` changes are intended to land	`TBD`	Cannot be inferred from the files alone	Ask the developer; see open question 1 in section 9
Real-world dataset size, group count and months kept	`TBD`	Original data was destroyed with the laptop wipe	Ask the teacher; no artefact survives to inspect
`LICENSE` copyright holder	`TBD`	Apache-2.0 template placeholder at `LICENSE:189` is still unfilled	Developer decision; inspect `LICENSE:189` after they fill it

2. Feature inventory

Feature	Trigger	Inputs	Output	Class
Group list with empty state	Page load (`index.html:409–422`, `1017–1039`)	none	Cards, or exactly `No groups yet. Click '+ Add Group' to get started!`	core
Group card summary	Render (`index.html:1041–1053`)	`group.name`, `group.dates.length`	`h2` name plus `{n} planned lessons`	core
Numeric-aware card sort	Render (`index.html:1029–1033`)	group names	`localeCompare` with `numeric:true`, `sensitivity:'base'`	incidental
Add group	`+ Add Group` (`index.html:232`, `488`, `603–637`)	name, price, currency	New group appended; title flips `Add Group` to `Edit Group`	core
Open group	Card click (`index.html:1044`)	array index	Group modal with info, month rows	core
Edit group info	Pencil `Edit group details` (`index.html:303`, `504`, `580–589`)	name, price, currency	Read-only display swaps to form	core
Currency choice	`#groupCurrencyInput` (`index.html:291–294`)	`UAH` or `PLN` only	Drives all `Intl` formatting	core
Save group	`Save` (`index.html:299`, `496–499`, `647–688`)	form values	Persists; also sets global `defaultCurrency`	core
Cancel info edit	`Cancel` (`index.html:298`, `500–503`, `590–601`)	none	Returns to display — **does not revert price**, see D2	core

Feature	Trigger	Inputs	Output	Class
Delete group	`Delete Group` (`index.html:342`, `707–716`)	confirm	Splices by index, persists, closes modal	core
Open schedule editor	`✏️ Edit Schedule` (`index.html:311`, `718–743`)	none	Calendar shown, month list hidden	core
Month-row deep link	Month row click (`index.html:1103–1107`)	month key	Calendar opens on that month	incidental
Toggle one date	Day cell click (`index.html:1161`, `783–791`)	date key	Adds or removes from `tempSelectedDates`	core
Weekday bulk select	Weekday header click (`index.html:1138`, `797–814`)	weekday index 0-6	Selects all, or clears all if already complete	core
Month and year navigation	`◄` `►`, select, year input (`index.html:318–321`, `816–843`)	month index, year number	Re-renders grid; **year unconstrained**, see D4	incidental
Jump to today	`Today` (`index.html:322`, `845–849`)	system clock	Resets to current month	incidental
Clear month	`Clear Month` (`index.html:323`, `851–856`)	none	Drops visible month's selections, no confirm	incidental
Bulk price for selection	`#selectedDatesPriceInput` (`index.html:331–332`, `858–881`)	number	Writes price to **every** month touched by the selection, see D3	core
Inline per-month price	`.month-price-input` during editing (`index.html:1094`, `1110–1115`)	number	Updates that month's price and total	core
Live selection summary	Any selection change (`index.html:1171–1173`)	count, price	`{n} days selected in {Month} — Total: {currency}`	incidental
Commit schedule	`Done` (`index.html:337`, `745–772`)	temp state	Persists dates and overrides; card count goes stale, see D6	core
Discard schedule	`Cancel` (`index.html:336`, `774–781`)	none	Drops temp state, no confirm	core
Monthly totals	Render (`index.html:1080–1101`)	lessons × price	`Total: {currency}`, `Per lesson: {currency}`	core

Feature	Trigger	Inputs	Output	Class
Generate payment message	`📋 Copy Payment Message` (`index.html:1099`, `1118–1122`, `1367–1378`)	template, month data	Review modal prefilled with substituted text	core
Edit message template	`📝 Edit Template` (`index.html:234`, `883–893`)	free text	Saved **globally** for all groups	core
Copy to clipboard	`Copy & Close` (`index.html:354`, `901–910`)	textarea value	Writes to clipboard; label shows `Copied!` for 1000 ms then closes	core
Export CSV	`Save CSV` (`index.html:236`, `915–931`)	in-memory groups	Download `lesson-planner-{UTC timestamp}.csv`	core
Import CSV	`Load CSV` plus hidden file input (`index.html:235`, `239`, `935–963`)	`.csv` file	**Replaces all data, no confirmation**	core
Clear all data	`Clear All Data` (`index.html:237`, `964–975`)	confirm	Clears groups and settings; **template survives**	core
Escape to close	`keydown` (`index.html:534–539`)	Escape	Closes topmost open modal, discarding edits	incidental
Overlay click to close	`click` (`index.html:506–508`)	click on backdrop	Closes **group modal only**	incidental
Unload warning	`beforeunload` (`index.html:541–546`)	any group exists	Browser's generic prompt; custom text ignored [S12]	vestigial
`App.utils.formatDate`	never called	—	—	vestigial
`groupInfoForm.onsubmit`	proven unreachable (`index.html:492–495`)	—	—	vestigial
`saveGroup` auto-save-schedule branch	proven unreachable (`index.html:673–675`)	—	—	vestigial

Views

All four views live in the DOM simultaneously; none is ever created or destroyed.

View	Lines	Visibility mechanism
Main screen: title, toolbar, group grid	229-243	always present

View	Lines	Visibility mechanism
Template modal	246-256	`.show` on `.modal-overlay`
Group modal: info display, info form, month rows, calendar	259-345	`.show`, plus `.hidden` and inline `display` internally
Review modal	348-357	`.show`

3. Data model

Three entities exist. There is **no student, no attendance, no payment record and no stable identifier anywhere.**

Entity	Fields	Types	Relationships	Volume
`Group`	`name`, `price`, `currency`, `dates`, `monthlyOverrides`	`string`, `number`, `"UAH"` or `"PLN"` in the UI but any string via CSV, `string[]`, object map	Root entity. Identity is its **array index** via `App.state.editingIndex`; CSV import re-keys by `name` (`index.html:1309`, `1323`) so duplicate names silently merge	281 bytes measured for 1 group with 5 dates and 2 months
`MonthlyOverride`	`price`, `dates`	`number`, `string[]`	Child of `Group`, keyed `YYYY-MM`. Created on demand (`ensureOverride`, `index.html:1227-1232`); deleted when its `dates` array is empty (`normalizeOverrides`, `index.html:1233-1241`)	about 34 bytes per month key plus 13 per date
`Settings`	`defaultCurrency`	`string`	Global singleton, not per group. Overwritten by the last saved group's currency (`index.html:683`) and by the **first** row of any CSV import (`index.html:1358`)	25 bytes measured
`Template`	the whole value	`string`, **not JSON**	Global singleton. Applies to every group and month. Absent until first edited, then falls back to `App.config.defaultTemplate`	446 characters for the substituted default message, measured

Cardinality and integrity notes

- `Group` 1:N `MonthlyOverride`. `Template` and `Settings` are 1:1 with the whole application.
- **`dates` is denormalised.** Every date string exists twice — once in `group.dates` and once in `monthlyOverrides[m].dates`. They are synchronised only in `saveDateChanges` (`index.html:745-772`) and in CSV import. `group.dates` drives the card count (`index.html:1045`); the override arrays drive all money arithmetic (`index.html:1086`). Divergence is silent.

- No schema version field exists on any entity.
- `Number(...) || 0` (`index.html:662`, `678`) means an empty or non-numeric price becomes `0`, not an error.
- Blank names get two different fallbacks: `'Untitled Group'` on create (`index.html:661`) but `'Untitled'` on edit (`index.html:677`).

Runtime-verified persisted shape

Read back from `localStorage` after a full add-group, paint-schedule, set-price, commit workflow:

```
[{"name":"Beginners A2","price":200,"currency":"PLN","dates":["2026-08-04","2026-08-11","2026-08-18","2026-08-25","2026-09-02"],"monthlyOverrides":{"2026-08":{"price":77,"dates":["2026-08-04","2026-08-11","2026-08-18","2026-08-25"]},"2026-09":{"price":77,"dates":["2026-09-02"]}}}]
```

Settings alongside it: `{"defaultCurrency":"PLN"}`.

Measured total for that dataset, key names included: **354 bytes**. *Inference:* ten groups over ten teaching months at four lessons each lands near 14 KB, so quota is not a practical concern; the real gap is the total absence of quota handling, not the volume.

4. Storage map

`localStorage` is the only persistence mechanism. Runtime-verified absent: `sessionStorage`, IndexedDB, cookies, service worker, Cache API, and any network write. Zero `<script src>`, zero `<link>`.

Storage key	API	Shape	Purpose	Versioned
`groupLessonPlannerData`	`localStorage`, `App.config.storageKey` (`index.html:377`)	`JSON.stringify(Array<Group>)`	All groups, dates, per-month prices	N
`groupLessonPlannerSettings`	`localStorage`, `App.config.settingsKey` (`index.html:378`)	`JSON.stringify({defaultCurrency})`	Last-used currency	N
`paymentTemplate`	`localStorage`, **bare string literal** at `index.html:1208` and `1211`	raw string, **not JSON**	Custom message template	N

Storage layer characteristics

- **No schema version, no migration path, no validation on load.** `load()` (`index.html:1188–1198`) calls `JSON.parse` on whatever it finds and assigns the result straight to `App.state.groups`. There is no shape check, so a hand-edited or partially-written value propagates directly into rendering.

- **No `try` / `catch` anywhere in the storage layer, and the failure mode is runtime-verified.** `this.services.storage.load()` runs at `index.html:411`, before the month dropdown is populated at 413-415, `this.bindEvents()` at 420 and `this.render.groups()` at 421. Planting the truncated value `[{"name":"Broken"` in `groupLessonPlannerData` and reloading produced: an uncaught `SyntaxError`, `cacheElements()` completed but `#monthSelect` left with **0 options**, `#groupList` innerHTML **empty with no empty-state element**, and `+ Add Group` inert because `bindEvents()` never ran. Nothing is shown to the user and no UI route recovers. The same absence of handling means `QuotaExceededError` on `save()` and the `SecurityError` some browsers raise in private-browsing modes are equally unguarded.
- **`clear()` is incomplete and the confirmation text overpromises.** `clear()` (`index.html:1203–1206`) removes only the two config keys. Runtime-verified: after confirming `Clear all groups and schedules? This cannot be undone.`, the remaining key set was exactly `["paymentTemplate"]` with a custom template still intact.
- **Two mutation paths never persist.** `save()` is called from `saveGroup` (684), `deleteGroup` (712), `saveDateChanges` (764) and `loadFromCsv` (948). It is **not** called by `updateDefaultPrice` (690-705) or `handleSelectedDatesPriceChange` (858-881), both of which mutate live state. Those edits survive in memory and get persisted later by an unrelated save, or vanish on reload.
- The `paymentTemplate` key is created only on first template save — runtime-verified absent on a fresh profile. Until then the hardcoded default at `index.html:382–400` supplies the value, which is why the personal data at 387-392 reaches every new user.
- Because `getTemplate()` uses `|| App.config.defaultTemplate` (`index.html:1208`), an empty-string value is falsy and silently falls back to the default. This accidentally neutralises the stray write described in D9.

5. Architecture notes

Concern	Current approach	Migration implication
Module system	One inline `<script>`, lines 359-1471, roughly 75 percent of the file. Zero imports, zero exports	Nothing is unit-testable in isolation today. Extracting `services` and `utils` first gives pure functions with no DOM coupling — the cheapest early win in a React migration
Global namespace	One `const App` at `index.html:361` with `state` 363, `config` 376, `elements` 404, `init` 409, `cacheElements` 427, `bindEvents` 486, `handlers` 552, `render` 978, `services` 1186, `utils` 1365	The seven-section split maps almost one-to-one onto a React structure: `state` to a store, `handlers` to actions, `render` to components, `services` to a persistence layer, `utils` to helpers
Global exposure	Runtime-verified: `window.App` is `undefined`, but a bare `App` resolves to an object with keys `state,config,elements,init,cacheElements,bindEvents,handlers,render,services,utils`. A top-level `const` in a classic script lives in the script scope, not on `window`	Tests can reach `App` via `page.evaluate` today. That backdoor disappears under a bundler, so any test relying on it must be rewritten against storage or the DOM before migration

Concern	Current approach	Migration implication
State ownership	Mutable shared object. Editing uses a copy-on-write pair, `tempSelectedDates` (a `Set`) and `tempMonthlyOverrides`, cloned via `JSON.parse(JSON.stringify(...))` (`index.html:1409–1411`)	The temp-copy pattern is sound and worth keeping. A `Set` does not survive `JSON.stringify`, so it must not leak into a serialised store
Rendering	Full destroy-and-rebuild on every change: `groupList.innerHTML=''` (1019), `monthlyOverrides.innerHTML=''` (1062), `calendar.innerHTML=''` (1131)	Semantically identical to React re-render, so behaviour will not change. All DOM identity, focus and scroll position is already lost on every update, so no regression risk there
DOM construction	Hybrid: template literals into `innerHTML` (1046, 1088), `insertAdjacentHTML` (1145), `createElement` plus `textContent` elsewhere	The `innerHTML` paths are the XSS sinks (D1). JSX escapes by default, so migration removes the class of bug rather than papering over it
Element lookup	All 47 ids cached once in `cacheElements()` (427–481). Runtime-verified: exactly 47 elements carry an `id`	A flat 47-entry element cache is the clearest signal that this is one component doing four jobs
Event wiring	About 32 `on*` **property assignments** plus exactly 3 `addEventListener` — overlay click (506), global keydown (534), `beforeunload` (541). Runtime-verified: zero inline `on*=` HTML attributes	Property assignment silently allows only one handler per event and is overwritten on each re-render. React's synthetic events remove the footgun
Modal visibility	`.show` class flipping `opacity` and `pointer-events` only, never `display` (`index.html:146–161`)	Root cause of the closed-modal accessibility defect (D8). The developer's uncommitted copy already fixes this with `hidden`; adopt that fix rather than reinventing it
Routing and URL	None. Zero use of hash, query, `history` or `pushState`	No deep links, no back-button semantics, nothing to preserve. Also means tests cannot navigate to a state — they must seed storage
Styling	One `<style>` block, lines 8–224. Custom property `--accent` only. About 6 dead rule groups and one duplicated rule	Small enough to port wholesale. Worth deleting the dead rules during the move, not before
Build tooling	**None.** No `package.json`, no bundler, no transpiler, no minifier in the deployed repo	Migration means introducing a toolchain from zero. `AGENTS.md` already documents npm scripts that do not exist in the committed tree — the docs describe the uncommitted future, not the present

Concern	Current approach	Migration implication
Third-party dependencies	**None.** Runtime-verified: 1 script tag, 0 with `src`, 0 `<link>`, 1 `<style>`. No CDN, no font URL, no `fetch`, no `XHR`	Nothing to audit for maintenance status; nothing to keep up to date. This is genuine simplicity worth not squandering
Browser APIs relied on	`localStorage`, `Intl.NumberFormat`, `navigator.clipboard`, `FileReader`, `Blob`, `URL.createObjectURL`, `Date`, `confirm`, `alert`	Only `navigator.clipboard` needs a secure context [S8]; the rest work on `file://`. `Intl` behaviour is the hidden landmine (D5)
Icons and typography	Inline Unicode emoji (📅 230, 🗒 234, ✏️ 303 and 311, 📋 1099, ◀ 318, ▶ 321). System font stack at line 9	Emoji render differently per platform, so pixel snapshots are not portable. Runtime-verified: the font stack **never reaches buttons** — they compute to `13.3333px Arial` because no `font: inherit` is set
Asset layout	One file. Runtime-verified 404s for `favicon.ico`, `robots.txt`, `sitemap.xml`, `manifest.json`, `sw.js`	No PWA, no offline story beyond the browser cache. A favicon is a one-line fix for a console error on every load

6. Non-functional baseline

Dimension	Observed state	Evidence
Requests after load	Exactly **one** document request. No second resource of any kind	`browser_network_requests` with static resources included returned a single `GET` returning 200
Page weight	58,649 bytes uncompressed, **13,610 bytes gzipped** on the wire	`content-length` with and without `Accept-Encoding: gzip`
Console cleanliness	**One error on every load**: `favicon.ico` 404, requested from the domain root rather than the project path	Console captured on the live site; root and project-path favicon both return 404
Load performance	Nothing to block: no external CSS, no external JS, no fonts, no images. Single round trip	Statically read plus the single-request measurement
Keyboard operability	**The primary workflow cannot be performed by keyboard.** Group cards compute `tabIndex: -1`; all 31 day cells and all 7 weekday headers compute `tabIndex: -1` with no `role`. The only focusables inside the calendar are its 9 chrome controls	Runtime-verified via computed `tabIndex` and a focusable-element sweep. Fails WCAG 2.2 SC 2.1.1 [S5]

Dimension	Observed state	Evidence
Tab order pollution	On a loaded page with one group and nothing open, the tab cycle has **14 stops, of which 9 are inside closed invisible modals**	Full tab sweep recorded: 5 toolbar buttons, then `templateTextarea`, `cancelTemplateBtn`, `saveTemplateBtn`, `editGroupInfoBtn`, `editScheduleBtn`, `deleteGroupBtn`, `reviewTextarea`, `cancelReviewBtn`, `copyAndCloseBtn`
Closed-modal semantics	Closed overlays compute `display: flex; opacity: 0; pointer-events: none; visibility: visible` — present in the accessibility tree	Computed style plus an accessibility snapshot showing headings `Edit Payment Message Template` and `Review Payment Message` and their controls while both modals were closed
Dialog semantics	Zero `role`, zero `<dialog>`, one `aria-*` attribute in the whole document, zero landmarks, no focus trap, no focus restoration	Runtime-verified counts: `roleAttrCount: 0`, `dialogElements: 0`, `ariaAttrCount: 1`, `landmarks: 0`
Accessible names	Three `<label>` elements have neither `for` nor a wrapped control (`index.html:267`, `271`, `275`). `#yearInput` (320) has no label. `◄` and `►` expose only a glyph. Weekday headers carry a `title` but no name or role	Runtime label audit plus statically read markup
Duplicate accessible names	With modals closed, the tree simultaneously exposes three buttons named `Cancel` and two named `Save`	Accessibility snapshot. Makes `getByRole('button', {name:'Cancel'})` ambiguous — a direct problem for the planned Playwright suite
Status announcements	No `aria-live` anywhere, so `#calendar-summary` and the `Copied!` label change silently	Statically read; `ariaAttrCount: 1` confirms the only `aria-*` is the pencil's label
Heading structure	`h1` then `h3` (skip), with `h4` inside the group modal and `h2` only in cards	Runtime-verified: `H1`, `H3 Edit Payment Message Template`, `H3 Edit Group`, `H4 Monthly Overrides & Schedule`, `H3 Review Payment Message`
Focus indication	Explicit ring exists **only** for `.icon-button:focus` (`index.html:134`). Every other control relies on the UA default	Statically read. Corrects the baseline notes, which reported this as a general focus ring
Target size	Day cells measure 79×29 px at desktop and 42×29 px at 375 px; toolbar buttons 29.5 px tall — all above the 24×24 minimum	Measured `getBoundingClientRect`. Passes WCAG 2.2 SC 2.5.8 [S13]
Colour: text contrast	White on accent `#4caf50` = **2.78:1**; danger `#ef4444` on white = **3.77:1**; muted `#94a3b8` on `#f8fafc` = **2.45:1**. All fail the 4.5:1 minimum [S3]. `#64748b` on `#f8fafc` = **4.55:1** passes, just	Ratios computed by me from runtime-resolved colours using the WCAG relative-luminance formula. Buttons are 13.33px so none qualifies as large text

Dimension	Observed state	Evidence
Colour: non-text contrast	`.today` border `#4caf50` on `#f8fafc` = **2.66:1**, below the 3:1 minimum [S4]. Card border `#e2e8f0` on body `#f8fafc` = **1.18:1**; card fill on body = **1.05:1**, so cards have essentially no visible boundary	Computed from runtime-resolved colours
Colour: weekend shading	**Exactly 1.00:1 — the weekend indicator is invisible.** Weekend cells resolve to `rgb(248,250,252)` and their container resolves to the identical `rgb(248,250,252)`; plain cells are `rgba(0,0,0,0)` and therefore render the same colour	Runtime-verified computed styles, confirmed visually in a 375 px screenshot. **Corrects the baseline notes**, which compared against white and called it "near-invisible"
Colour as sole channel	Date selection is conveyed only by green fill plus weight; today only by border colour; weekends by nothing at all	Statically read `index.html:177–178`, `191–193`. Fails WCAG 2.2 SC 1.4.1 [S14]
Responsive: main screen	No horizontal page overflow at 375 px; the toolbar wraps inside a 129 px tall `h1`, the same height measured at desktop width, so this is graceful rather than a mobile-specific break	Measured `scrollWidth` equal to `clientWidth` at 375 px, and identical `h1` height at 375 px and 1280 px
Responsive: modals	375 px wide, `max-height` 730.8 px (90vh), `overflow: auto` — scrolls correctly	Measured
Responsive: calendar breaks	`.calendar-controls` computes `flex-wrap: nowrap`, so at 375 px `#clearMonthBtn` extends to x=394 in a 375 px viewport and the `◂` button is clipped at the left edge. Both are unreachable without scrolling inside the modal	Measured overflow plus a 375 px screenshot showing `◂` half-cut and `Clear Month` truncated
Responsive: summary clipping	`#calendar-summary` has a fixed `height: 1em` (12 px) while its content needs 14 px, clipping 2 px	Measured `scrollHeight: 14` against `clientHeight: 12`
Time dependence	`new Date()` at module init (369-370), today highlighting (1142, 1156), `jumpToToday` (846), schedule-editor default month (732-734), the current-month cutoff in `updateDefaultPrice` (696-699), and the export filename (923)	Statically read. Tests must freeze the clock; `playwright.config.ts` already pins `timezoneId: 'UTC'`
Timing constants	`setTimeout` delays of 0 (588), 100 (634, 886, 898) and 1000 ms (909), plus CSS transitions of 0.2-0.3 s (29, 100, 156) that race visibility because modals never change `display`	Statically read; the 1000 ms window and the `Copied!` transition were runtime-observed

Terminology and localisation

`<html lang="en">` at `index.html:2`. All chrome is English. Month, short-day and full-day names are hardcoded English arrays (`index.html:379–381`) and every format call is pinned to `en-US` (`1373`, `1380`, `1386`).

The exception is the default payment template (`index.html:382–400`), which is **mixed English and Ukrainian** — runtime-verified as 6 Cyrillic lines out of 19, using Ukrainian banking vocabulary, typographic guillemets `«»` and curly quotes. `lang="en"` is therefore wrong for that block, and a screen reader will mispronounce it. No Polish or Russian text exists; `PLN` appears only as a currency option (`index.html:293`).

Domain terms that must survive a rewrite unchanged: **Group**, **Default Price**, **Currency**, **Monthly Overrides**, **Schedule**, **planned lessons**, **Per lesson**, **Total**, **Payment Message**, **Template**, the month key format `YYYY-MM`, the date key format `YYYY-MM-DD`, and the three template tokens `{{month}}`, `{{lessons}}`, `{{total}}`.

7. Observable-behaviour surface usable as test anchors

Everything below is present in the **deployed** file. The deployed file has zero `data-testid` attributes, so none is listed.

Anchor	Exact value or selector	Verification
Empty state	`No groups yet. Click '+ Add Group' to get started!`	runtime-verified
Card lesson count	`{n} planned lessons`	runtime-verified: `Beginners A2 5 planned lessons`
Month row heading	`{MonthName} {Year}` plus `({n} lessons)`	runtime-verified: `August 2026 (4 lessons)`
Month totals	`Total: {formatted}` and `Per lesson: {formatted}`	runtime-verified: `Total: PLN 308.00`, `Per lesson: PLN 77.00`
Currency format	`Intl.NumberFormat('en-US', {style:'currency', currency})` renders `UAH 1,234.50` and `PLN 1,234.50` — code prefix plus space, not a symbol	runtime-verified
Calendar summary	`{n} days selected in {Month} — Total: {formatted}`, with a real **em dash**	runtime-verified: `4 days selected in August — Total: UAH 600.00`
Weekday header tooltip	`Select all {FullDayName}s in this month`	runtime-verified for all seven
Copy feedback	Button label becomes `Copied!` then reverts after 1000 ms	runtime-verified with timed sampling

Anchor	Exact value or selector	Verification
Modal titles	`Add Group`, `Edit Group`, `Edit Payment Message Template`, `Review Payment Message`, `Monthly Overrides & Schedule`	runtime-verified
Pencil accessible name	`Edit group details` (both `title` and `aria-label`)	runtime-verified
Delete confirm	`Delete group "{name}"?`	runtime-verified: `Delete group "Beginners A2"?`
Clear-all confirm	`Clear all groups and schedules? This cannot be undone.`	runtime-verified
Empty-export alert	`There are no groups to export yet.`	runtime-verified
CSV error alerts	`Unable to load CSV: {message}` wrapping `CSV file is empty.`, `Missing "{col}" column in CSV.`, `Invalid month format: "{v}"`, `Invalid month value: "{v}"`, `Malformed CSV: unmatched quote detected.`	four runtime-verified, the unmatched-quote message statically read at `index.html:1448`
Unload prompt	`beforeunload` fires whenever a group exists; the browser supplies its own text and discards the app's [S12]	runtime-verified — dialog surfaced with an empty message
Persisted state	Three keys, exact JSON shape in section 3. `paymentTemplate` is a **raw string, not JSON**	runtime-verified
Clipboard	One write, sourced from `#reviewTextarea.value`; requires a secure context [S8]	runtime-verified: 446 characters landed on the clipboard
Download	Blob then object URL then synthetic `<a download>`; filename `lesson-planner-{UTC ISO with colons and T replaced by hyphens}.csv`	runtime-verified: `lesson-planner-2026-08-20-16-43-14.csv`
CSV header row	`"Name","Default Price","Currency","Month","Month Price","Dates"` — every field quoted, CRLF row endings, **no UTF-8 BOM**	runtime-verified from the downloaded bytes: first bytes were `22 4e 61 6d 65 22`, not `EF BB BF`
CSV date column	Space-delimited ISO dates within one quoted field	runtime-verified
Stable ids	All 47, cached at `index.html:427-481`	runtime-verified count
Stable classes	`.group-card`, `.group-card-info`, `.month-override-row`, `.month-name`, `.month-total`, `.price-per-lesson`, `.month-price-input`, `.copy-msg-btn`, `.day`, `.selected`, `.today`, `.weekend`, `.spacer`, `.empty-state`, `.hidden`, `.show`	statically read; no hashed or generated names exist

Anchor	Exact value or selector	Verification
Isolation properties	Zero network requests, zero URL or hash or history usage, zero cookies, zero external assets. State can be seeded entirely by writing the three keys before load	runtime-verified

Anti-anchors — do not build tests on these. Modal open state is *not* observable via `display` or `visibility`, since closed modals compute `display: flex; visibility: visible`; only the `.show` class and `opacity` distinguish them. Role-based name queries for `Cancel` and `Save` are ambiguous because closed modals stay in the tree. And `App` is reachable from `page.evaluate` today but will not survive bundling.

8. Defects, assumptions, and hardcoded values

Defects

#	Item	Location	Evidence	Risk
D0	**Personal data hardcoded in source.** A real person's full name, bank IBAN and tax identification number sit in the default payment template, plus bank identifiers and a personal first name in the signature. Publicly readable via view-source on the live site, and served to every new user who never edits the template	`index.html:387–392`, `400`; template block `382–400`	statically read; runtime-verified as 6 Cyrillic lines reaching the generated message	**Critical.** Live exposure of financial and tax identifiers. Present since the app's first commit per the orchestrator's git read, so removing it from `HEAD` does not remove it from history
D1	**Stored XSS.** `group.name` is interpolated unescaped into `innerHTML`. A CSV-supplied name containing an `<img>` tag with an `onerror` attribute parsed into a real DOM element and **executed**; the payload persists to `localStorage` and re-fires on every load	sink `index.html:1046–1051`; second sink `1088–1101`; source `1313`	runtime-verified: marker variable set to `true`, `IMG` element confirmed in the DOM	**High.** Untrusted-file channel to script execution. JSX escaping removes this class of bug on migration
D2	**Cancel does not revert the default price.** `groupPriceInput.onchange` mutates `group.price` immediately; `cancelGroupInfoEdit` then re-reads the already-mutated state	`index.html:509`, `690–705`, `590–601`	runtime-verified: typed 999, clicked Cancel, state and display both read 999	High. Silent unintended price change, and it cascades into future-month overrides

#	Item	Location	Evidence	Risk
D3	**Cross-month price bleed.** The bulk price input writes to **every** month touched by the selection, while its own value and enabled state derive only from the visible month. Selections survive month navigation	`index.html:858–881` against `1176–1181`	runtime-verified: typed 77 with only September visible; August's price silently became 77 and its total became `PLN 308.00`	**High.** Rewrites money in months the user cannot see, with no indication
D4	**Unconstrained year input corrupts date keys and breaks CSV restore.** `#yearInput` has no `min`, `max` or `step`. Year `5` yields `App.utils.iso` keys like `5-08-10`; `toMonthKey` is a naive `slice(0,7)` so the month key becomes a full date. Re-importing the app's own export then throws	`index.html:320`, `823–828`, `1394–1396`, `1398–1401`, `1457–1466`	runtime-verified: persisted `"5-08-10"` as both a date and a month key; re-import threw `Invalid month format: "5-08-10"`; blank yields year 0 and negatives are accepted	**Critical.** The app can create data that permanently breaks its only backup path, and produces two indistinguishable `August 5` rows
D5	**A non-3-letter currency makes a group permanently unopenable.** CSV import accepts any non-empty string as currency. `Intl` accepts any 3 ASCII letters but throws `RangeError` otherwise. The throw happens inside `render.monthlyOverrides()`, which runs **before** `classList.add('show')`, so `openGroupModal` aborts midway	source `index.html:1317`; sink `1379–1381` via `1091`; abort order `628` before `632`	runtime-verified: imported currency `US Dollar`; clicking the card threw `Invalid currency code : US Dollar`, the modal never opened, month rows were emptied, and the previous group's name was left stranded in the display	**Critical.** No UI route to open, fix or delete the group. Only Clear All Data or a fresh import escapes
D6	**Group card lesson count goes stale after Done.** `saveDateChanges` re-renders month rows, group info and the calendar but never calls `render.groups()`	`index.html:745–772`, missing a `render.groups()` call	runtime-verified: card read `0 planned lessons` with five dates persisted; a reload corrected it to `5 planned lessons`	Medium. Persisted data is correct; the user is shown a wrong number and may re-do work

#	Item	Location	Evidence	Risk
D7	**Committing a price change silently discards an unsaved name edit.** The price `change` handler calls `render.groupInfo()`, which unconditionally rewrites all three inputs from saved state	`index.html:509` to `704`, overwriting at `1008–1010`	runtime-verified: typed `Renamed Group XYZ`, tabbed out of the price field, and the name input reverted to `Beginners A2`	**High.** Data loss on the most natural editing path — rename and reprice in one visit. Same mechanism can discard a currency selection
D8	**Closed modals stay focusable and screen-reader visible**, and activating their controls has real effects. Visibility is `opacity` and `pointer-events` only, never `display`	`index.html:146–161`	runtime-verified: 9 of 14 tab stops sit in closed modals; computed `visibility: visible`	**High.** Fails WCAG 2.2 SC 2.1.1 and 4.1.2 [S5][S6]. Already fixed in the developer's uncommitted copy
D9	Activating those invisible controls: the invisible `✏️ Edit Schedule` throws an **uncaught page error** `No group is currently being edited.`; the invisible template `Save` writes an empty string to `paymentTemplate`; the invisible `Copy & Close` overwrites the clipboard and shows `Copied!`	`index.html:558`, `889–893`, `901–910`	runtime-verified all three	Medium. The empty-string write is masked by the `
D10	**Calendar and cards are not keyboard operable.** Group cards, 31 day cells and 7 weekday headers are `div`s with `onclick`, `tabIndex: −1`, no `role`, no key handler	`index.html:1042–1044`, `1134–1138`, `1150–1161`	runtime-verified computed `tabIndex` and focusable sweep	**High.** The core task — painting dates and opening a group — is mouse-only. Fails SC 2.1.1 [S5]
D11	**CSV import replaces all data with no confirmation**, unlike Clear All Data which asks	`index.html:946–948`	runtime-verified: two groups replaced by imported content, zero dialogs	High. Destructive and irreversible, guarding the app's only restore path
D12	**A stray quote that happens to balance is silently accepted and destroys existing data.** The parser throws only for a quote left open at end of file	`index.html:1413–1455`, guard at `1447–1449`	runtime-verified: a mis-quoted row replaced both real groups with one garbage group named `Unmatched quote`, with **no dialog**. Empty file, missing header and bad month all correctly threw and preserved data	**High.** Contradicts the prior-art claim that malformed CSV throws

#	Item	Location	Evidence	Risk
D13	**Escape discards pending schedule and price edits with no confirmation**	`index.html:534–539` to `639–645` to `574–579`	runtime-verified: two added dates and a 150-to-999 price change vanished, zero dialogs	Medium
D14	**No storage error handling at all.** A corrupt `groupLessonPlannerData` value aborts `App.init()` mid-way, leaving a page that renders no groups, no empty state, an empty month dropdown, and buttons that do nothing because `bindEvents()` never ran	`index.html:1188–1213`, order set by `409–422`	runtime-verified: planted a truncated JSON value, reloaded, observed the uncaught `SyntaxError`, 0 month options, empty `#groupList`, and an inert `+ Add Group`	**Critical.** Silent, total, and unrecoverable through the UI. Most plausible mechanism for a repeat of the original data loss
D15	**`clear()` leaves `paymentTemplate` behind** while the confirm text promises a full wipe	`index.html:1203–1206` against `965`	runtime-verified: remaining keys were exactly `["paymentTemplate"]`	Low, but it is exactly the key holding the personal data
D16	**CSV export omits the template**, so the advertised backup does not back up the message text	`index.html:1251–1282`	statically read plus runtime-verified export columns	Medium. Silent partial backup
D17	**No UTF-8 BOM on export**, so Cyrillic group names mojibake in Excel on Windows	`index.html:921`	runtime-verified from downloaded bytes	Medium for a user whose names are Cyrillic
D18	**Unhandled clipboard promise.** `writeText` is neither awaited nor caught, yet `Copied!` and the close are unconditional, so the success indicator can lie. No fallback	`index.html:901–910`	statically read; the success path was runtime-verified to genuinely copy	Medium. Silent failure outside a secure context
D19	**Pluralisation is broken.** `1 days selected in September`, `(1 lessons)`	`index.html:1089`, `1172`	runtime-verified both strings	Low, but user-visible on every single-lesson month

#	Item	Location	Evidence	Risk
D20	`normalizeOverrides` deletes any override whose `dates` array is empty, so a price set before dates are chosen does not survive a save. Note the guard is `data.dates && data.dates.length === 0`, so an override with no `dates` **property** survives	`index.html:1233–1241`	statically read	Medium. Silent loss of intent
D21	`#calendar-summary` has a fixed `height: 1em` and clips its own text	`index.html:194–200`	runtime-verified: `scrollHeight` 14 against `clientHeight` 12	Low
D22	`.calendar .spacer` shows `cursor: pointer` but has no handler — a lie about affordance	`index.html:176`	runtime-verified computed `cursor: pointer` on 5 spacers	Low
D23	Favicon 404 logs a console error on every load, requested from the domain root not the project path	no favicon in repo	runtime-verified on the live site	Low
D24	`beforeunload` fires on every navigation once any group exists, and browsers discard the custom text, so its CSV-backup instructions are never seen [S12]	`index.html:541–546`	runtime-verified: dialog fired with an empty message	Low, but it is pure friction with no payoff
D25	Month name is derived two different ways: `config.monthNames` for the UI, `toLocaleString` for the message	`index.html:379` against `1373`	statically read	Low. Two sources of truth that can diverge

Assumptions, hardcoded values, magic strings, dead code

Item	Location	Risk
Group identity is its **array index**; CSV import re-keys by **name**	`index.html:604`, `1309`, `1323`	High. Duplicate names silently merge on import; any reorder invalidates a held index
`'UAH'` hardcoded in five places	`index.html:371`, `967`, `1196`, `1317` (twice on the line), `1358`	Medium. The app is implicitly Ukraine-first while offering PLN

Item	Location	Risk
Currency whitelist exists **only** as `<option>` markup, so `render.groupInfo` reads the DOM to learn the domain rule	`index.html:292–293` read at `1001–1002`	Medium. Business rule embedded in markup; also the source of the D5 display split, where the same screen showed `UAH` in the header and `XYZ` in month rows
`'en-US'` hardcoded three times	`index.html:1373`, `1380`, `1386`	Medium. Blocks localisation for a Ukrainian and Polish audience
Two different blank-name fallbacks	`'Untitled Group'` at `661` against `'Untitled'` at `677`	Low. Inconsistent data
`new Date(monthKey + '-02')` day-02 trick to dodge timezone rollback	`index.html:1373`	Low. Correct, but unexplained and fragile to refactoring
Weekday maths `(getDay() + 6) % 7` repeated three times	`index.html:803`, `1154`, `1389`	Low. Monday-first convention duplicated
Escape handler builds method names by string concatenation	`index.html:538`	Medium. Silently breaks under minification or a rename
Behaviour keyed off an inline style **string** comparison	`index.html:673`	Medium. Reads presentation to decide logic; part of the unreachable branch below
Timing constants 0, 100, 100, 100, 1000 ms	`index.html:588`, `634`, `886`, `898`, `909`	Low. Test flakiness source
No `min`, `max`, `step`, `required` or `maxlength` on any input	`index.html:282`, `287`, `320`, `332`	Medium, and the direct cause of D4. Runtime-verified all null
Dead: `App.utils.formatDate` never called	`index.html:1382–1387`	Low. Single grep hit confirms
Dead: `groupInfoForm.onsubmit` unreachable	`index.html:492–495`	Low. **Runtime-verified:** pressing Enter in the name field produced `saveGroupCalls: 1` and `submitFired: 0`, because implicit submission clicks the default button [S10] whose handler calls `preventDefault()`
Dead: `saveGroup`'s auto-save-schedule branch unreachable	`index.html:673–675`	Medium, because prior art depends on it. **Runtime-verified:** while the calendar is open, `#groupInfoForm` computes `display: none`, `#saveGroupBtn` has a 0×0 rect and a null `offsetParent`, and `#editGroupInfoBtn` is also `display: none` and unclickable even with a forced click

Item	Location	Risk
Dead CSS: `.group-dates` and `.group-dates.show`, `.date-badge`, `.payment-message` and its `pre`, `.payment-actions`, `.modal-summary`, `.selected-dates-list`	`index.html:91-107`, `117-126`, `203-224`	Low. About 35 lines, verified by whole-file occurrence counts
Duplicate CSS rule: `.month-override-row .month-total` declared twice	`index.html:70-72` and `81-85`	Low
Redundant: `groupInfoDisplay.style.opacity` set alongside the `.hidden` class that already sets `display: none`	`index.html:1012-1013`	Low
Current calendar month always gets a row even with zero lessons	`index.html:1065-1066`	Low. Confusing empty row with a disabled button
`loadFromCsv` body is dedented out of its enclosing `try` while remaining syntactically inside it	`index.html:946-954`	Low. Hand-patch smell that misleads readers about the error boundary

Prior-art claims found to be false or misleading

The following are in **uncommitted local work** in the main checkout. They are cited as evidence of intent, never as authority.

Claim	Source	Finding
"Save while calendar editor is still open... Date edits are persisted because `saveGroup` auto-calls `saveDateChanges`", rated **P0**	`docs/qa-coverage-investigation.md`, scenario LP-010	**False.** Runtime-verified that no UI path reaches `saveGroup` while the calendar is open: the form and the pencil are both `display: none`. A P0 test would be written against unreachable code
"Throws on missing columns, malformed CSV, or invalid month values"	`docs/tech-details.md`, CSV import behavior	**Partly false.** Empty file, missing header and invalid month all throw and preserve data, but a stray quote that balances is silently accepted and destroys existing data (D12)
"The UI exposes stable `data-*` hooks for Playwright"	`docs/tech-details.md`, Test Hooks	**Not true of the deployed app.** Runtime-verified zero `data-*` and zero `data-testid` attributes. The hooks exist only in the uncommitted `index.html`
Line references such as `index.html:1195-1218` for the storage service	`docs/qa-coverage-investigation.md` throughout	**Off by roughly 8-18 lines.** They resolve against the 1491-line working copy; the storage service is at 1187-1213 in the deployed file

Claim	Source	Finding
`AGENTS.md` documents `npm install`, `npm run serve`, `npm run test:e2e`	`AGENTS.md:9–15`, committed	No `package.json` exists in the committed tree. The committed guidelines describe the uncommitted future
"`npm run test:e2e` passed 22/22 on March 20, 2026"	`docs/qa-coverage-investigation.md`	Not reproduced by me. Plausible, but it was run against the modified `index.html`, so it does not evidence the deployed app

Where the orchestrator's baseline notes were wrong

Baseline claim	Correction
Prior art is limited to uncommitted tests and docs	**`index.html` itself is modified in the main checkout** (1491 lines, 59,604 bytes), adding `hidden` modal handling and 14 kinds of test hook. The most consequential omission
Muted `#94a3b8` on white is "~2.3:1"	**2.45:1** on the `#f8fafc` surface where it actually renders, or 2.56:1 on white. Both fail, but the figure was wrong
`.weekend` `#f8fafc` versus white is "near-invisible"	It renders on a `#f8fafc` container, not white, giving **exactly 1.00:1**. Not near-invisible — invisible
"Explicit focus ring (134)" listed as a general positive	Scoped to `.icon-button:focus` only. Every other control uses the UA default
Personal data "at `index.html:386–392`"	Line 386 is the heading `Payment details:`. The name is at 387, IBAN 388, tax identifier 389, bank identifiers 390-392, and a personal first name at **400**, which the notes missed
`groupInfoForm.onsubmit` unreachable, and the RangeError and closed-modal focus issues, all flagged as inference to verify	All now **runtime-verified**. The inferences were correct
CSV import "accepts any string as currency" so `formatCurrency` "throws on an invalid ISO code"	Narrower and worse. `Intl` accepts **any 3 ASCII letters**, including unassigned `XYZ`, and lowercase. It throws only for non-3-letter values — but when it does, the group becomes permanently unopenable (D5), which the notes did not identify
Volume "well under 100 KB", quota "not a practical risk"	Direction right, basis absent. Measured **354 bytes** for 1 group with 5 dates and 2 months. The quota itself is `TBD`
Zero external requests after load	Correct for resources, but there **is** a favicon request that 404s and logs a console error on every load
Defect list of 18 items	Five were missing, two of them data-loss class: D7 name-edit reversion, D6 stale card count, D5 unopenable group, D12 silent malformed-CSV data destruction, D19 pluralisation

9. Open questions requiring code-level inspection

1. **Is the uncommitted `index.html` in the main checkout intended to land?** It fixes D8 and supplies the test hooks the docs already describe. If yes, it should be committed before any baseline is frozen, because it changes the a11y posture, the tab order and the entire test-anchor surface. If no, the four `docs/*.md` files must be corrected — their line numbers and their Test Hooks section describe code that is not deployed. *This blocks any downstream report that cites line numbers or plans DOM-level test anchors.*
2. **Was the CSV round-trip ever exercised with real data?** The export omits `paymentTemplate` (D16), has no BOM (D17), and one malformed date makes the whole restore throw (D4). If the teacher's actual backup habit was CSV, the restore may never have been proven to work. Needs a conversation, not code.
3. **Is the cross-month bulk price (D3) intended?** `docs/qa-coverage-investigation.md` itself asks "Should bulk price change only the visible month or every selected month?" — so the developer does not know either. This is a product decision that must be settled before it is encoded in tests. Downstream pricing work is blocked until it is.
4. **Should `Clear All Data` also clear `paymentTemplate`?** The confirm text says it will. Given the template holds the personal data (D0), the privacy answer and the correctness answer point the same way.
5. **What should happen to `defaultCurrency` as a global?** It is currently overwritten by the last saved group and by the first row of any import. If groups can legitimately differ in currency, a single global is the wrong shape.
6. **Is `beforeunload` (D24) wanted at all?** Data is already auto-saved, browsers discard its message, and it fires on every navigation. The prior-art doc raises the same doubt.
7. **What is the real dataset scale?** Needed to size any storage migration. Currently unanswerable — the data is gone.
8. **Does the developer want to keep the two-array denormalisation of `dates`?** It is the main invariant a typed model would have to either enforce or eliminate, and D6 is a symptom of it.

10. Sources

#	Title	URL	Accessed	Supports
S1	MDN — Window.localStorage	https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage	2026-08-20	localStorage is synchronous, origin-scoped and stores strings only, so all structure must be serialised
S2	MDN — Web Storage API	https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API	2026-08-20	Storage limits are browser-dependent, which is why the quota is recorded as TBD rather than estimated

#	Title	URL	Accessed	Supports
S3	WCAG 2.2 — Understanding SC 1.4.3 Contrast (Minimum)	https://www.w3.org/WAI/WCAG22/Understanding/contrast-minimum.html	2026-08-20	The 4.5:1 threshold and the large-text exemption used to judge the 2.78:1, 3.77:1 and 2.45:1 measurements
S4	WCAG 2.2 — Understanding SC 1.4.11 Non-text Contrast	https://www.w3.org/WAI/WCAG22/Understanding/non-text-contrast.html	2026-08-20	The 3:1 threshold for UI component boundaries, failed by the today border at 2.66:1 and the card border at 1.18:1
S5	WCAG 2.2 — Understanding SC 2.1.1 Keyboard	https://www.w3.org/WAI/WCAG22/Understanding/keyboard.html	2026-08-20	All functionality must be keyboard operable, the criterion failed by non-focusable cards and day cells
S6	WCAG 2.2 — Understanding SC 4.1.2 Name, Role, Value	https://www.w3.org/WAI/WCAG22/Understanding/name-role-value.html	2026-08-20	Custom controls need a programmatic role and name, absent from every `div`-based control here
S7	GitHub Docs — Configuring a publishing source for your GitHub Pages site	https://docs.github.com/en/pages/getting-started-with-github-pages/configuring-a-publishing-source-for-your-github-pages-site	2026-08-20	Branch-based publishing needs no in-repo workflow, which is why no build config exists yet the site deploys
S8	MDN — Clipboard.writeText()	https://developer.mozilla.org/en-US/docs/Web/API/Clipboard/writeText	2026-08-20	Returns a Promise and requires a secure context, the basis for D18 and for testing over http://127.0.0.1
S9	MDN — Intl.NumberFormat() constructor	https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/NumberFormat/NumberFormat	2026-08-20	A currency code must be three alphabetic characters or the constructor throws RangeError, explaining D5
S10	HTML Standard — Implicit submission	https://html.spec.whatwg.org/multipage/form-control-infrastructure.html#implicit-submission	2026-08-20	Enter activates the default button rather than submitting directly, which is why `onsubmit` never fires
S11	MDN — Date() constructor	https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/Date	2026-08-20	Years 0 to 99 map into 1900-1999, the mechanism behind year `5` rendering a 1905 grid in D4

#	Title	URL	Accessed	Supports
S12	MDN — Window beforeunload event	https://developer.mozilla.org/en-US/docs/Web/API/Window/beforeunload_event	2026-08-20	Browsers show their own generic string and ignore custom text, confirming D24
S13	WCAG 2.2 — Understanding SC 2.5.8 Target Size (Minimum)	https://www.w3.org/WAI/WCAG22/Understanding/target-size-minimum.html	2026-08-20	The 24×24 CSS-pixel minimum that the measured 42×29 and 79×29 targets satisfy
S14	WCAG 2.2 — Understanding SC 1.4.1 Use of Color	https://www.w3.org/WAI/WCAG22/Understanding/use-of-color.html	2026-08-20	Colour must not be the only means of conveying information, failed by selection, today and weekend styling

No third-party libraries were found in the source, so no library maintenance status is reported. Runtime-verified: 1 `<script>` with no `src`, 0 `<link>` elements, no `@import`, no CDN reference and no network request beyond the document itself.

11. Quick wins

Every item below is user-visible or privacy-critical, small, low-regression, and blocked by no open decision in this report. Deliberately excluded: fixing D3 cross-month price bleed (blocked by open question 3), adopting the uncommitted `index.html` wholesale (blocked by open question 1), and clearing `paymentTemplate` in `clear()` (blocked by open question 4).

Rank	Quick win	Effort	Impact	Basis of ranking
1	Remove hardcoded personal data from the default template	XS	Critical	Live exposure of a real IBAN and tax identifier via view-source. One-block edit, no logic change, no storage change. Nothing else in this report outranks it
2	Hide closed modals with `hidden` so they leave the tab order	S	High	Removes 9 of 14 bogus tab stops and the D9 side effects in one change. The developer has already written and exercised this exact fix, so the design risk is zero
3	Constrain the year input	XS	High	Three attributes plus a clamp stop the app from creating data that breaks its own CSV restore (D4). Purely additive validation
4	Re-render the group list after a schedule save	XS	Medium	One added line fixes a wrong number shown after the most common workflow (D6). No behaviour beyond the refresh changes
5	Fix pluralisation in the two count strings	XS	Medium	Every single-lesson month currently reads `1 lessons`. Pure string formatting, and it settles the exact test-anchor wording before a suite is written

Rank	Quick win	Effort	Impact	Basis of ranking
6	Add a UTF-8 BOM to the CSV export	XS	Medium	One character makes Cyrillic group names open correctly in Excel on Windows (D17), for a user whose names are Cyrillic. Round-trip safety confirmed: the header lookup lowercases a trimmed cell at `index.html:1289`, and `String.prototype.trim` strips U+FEFF, so `name` still resolves

```
PROMPT QW-1: Remove hardcoded personal data from the default payment template
Context: Repo lesson-planner, single deployed file index.html at the repo root, served by
GitHub Pages. App.config.defaultTemplate at index.html:382-400 is the fallback payment
message used whenever the localStorage key paymentTemplate is absent, which is the case for
every new user. Lines 387-392 contain a real person's full name, bank IBAN, tax
identification number and bank identifiers; line 400 contains that person's first name as a
signature. All of it is publicly readable via view-source on the live site.
Task: Replace the payment-details block at index.html:387-392 and the signature at
index.html:400 with neutral placeholder text that tells the user to enter their own details,
keeping the surrounding English structure, the three template tokens {{month}}, {{lessons}}
and {{total}}, and the Ukrainian payment-purpose line at index.html:395 intact. Do not
reproduce any of the removed values anywhere in the repo, including comments, docs and commit
messages.
Constraints: Do not rename or restructure App.config.defaultTemplate. Do not change the three
localStorage key names groupLessonPlannerData, groupLessonPlannerSettings and
paymentTemplate, and do not change the persisted data shape. Do not touch the getTemplate
fallback logic at index.html:1207-1209, so a user who already saved a custom template keeps
it untouched. No new dependencies, no build step. Out of scope: purging the values from git
history, which is a separate decision the developer must make.
Acceptance criteria: All four of these greps over index.html return zero matches, run with
grep -E: the IBAN shape [A-Z]{2}[0-9]{6,}; any digit run of 8 or more [0-9]{8,}; any Cyrillic
character on lines 382 to 394 or line 400 of the template block, tested with the class
[\x{0400}-\x{04FF}] using grep -P; and the fixed strings IBAN and MFO case-insensitively. The
one intentional Cyrillic line, the payment-purpose text at index.html:395, is the sole
permitted exception and must remain byte-identical. The default template still contains
{{month}}, {{lessons}} and {{total}} exactly once each. With localStorage empty, creating a
group with one lesson and clicking Copy Payment Message yields a message with all three
tokens substituted and no remaining brace characters. A user with an existing paymentTemplate
value still sees that value in the Edit Template modal.
Verification: Run the four greps above and confirm each returns nothing. Then serve the file
locally, clear localStorage, add a group, select one date, click Done, click Copy Payment
Message, and read the review textarea. Finally set paymentTemplate to a custom string,
reload, open Edit Template, and confirm the custom string is shown unchanged.
```

```
PROMPT QW-2: Take closed modals out of the tab order and the accessibility tree
Context: Repo lesson-planner, index.html. The three modal overlays templateModal, groupModal
and reviewModal at index.html:246, 259 and 348 are shown and hidden purely by adding and
removing a .show class that toggles opacity and pointer-events (CSS at index.html:146-161).
display stays flex and visibility stays visible, so closed modals remain focusable and
exposed to screen readers. Measured on a loaded page with one group: 9 of the 14 tab stops
belong to closed modals, and activating them causes an uncaught error from index.html:558, a
stray empty write to paymentTemplate, and a silent clipboard overwrite. An uncommitted
working copy of index.html in the developer's main checkout already implements the intended
fix; mirror that approach.
Task: Add the hidden attribute to all three modal overlay elements in the markup, add a CSS
```

rule [hidden] { display:none !important; } next to the existing .hidden rule at index.html:143, and in every open and close handler set the overlay's hidden property to false before adding .show and to true after removing .show. The handlers to update are openGroupModal (index.html:603–637), closeGroupModal (639–645), openTemplateModal (883–887), closeTemplateModal (888), openReviewModal (895–899) and closeReviewModal (900).
Constraints: Keep the .show class and the opacity transition so the fade-in is preserved. Keep the Escape handler at index.html:534–539 working, including its detection of the open modal via classList.contains('show'). Do not change the three localStorage key names or the persisted data shape. Do not add roles, aria-modal, a focus trap or focus restoration in this change; those are a larger accessibility task. No new dependencies.
Acceptance criteria: On a freshly loaded page with one existing group and no modal open, tabbing from the top reaches exactly the five toolbar buttons and then leaves the document, with zero stops inside any modal. Opening each of the three modals still shows it, and its controls are focusable while it is open. Pressing Escape still closes an open modal. Closed overlays report a computed display of none.
Verification: Serve locally, seed one group, then tab from the document start and record each document.activeElement id; assert the sequence is addGroupBtn, editTemplateBtn, loadCsvBtn, saveCsvBtn, clearDataBtn and then no modal control. Open and close each modal once to confirm normal operation, and confirm no uncaught error appears in the console.

PROMPT QW-3: Constrain the calendar year input so it cannot corrupt date keys
Context: Repo lesson-planner, index.html. The year input at index.html:320 is type=number with no min, max or step. updateCalendarYear at index.html:823–828 does Number(value) and stores it unvalidated in App.state.calYear. App.utils.iso at index.html:1394–1396 pads only the month and day, so a year of 5 produces date keys like 5-08-10, and App.utils.toMonthKey at index.html:1398–1401 is a naive slice(0,7) that then yields a month key of 5-08-10 rather than a year and month. Verified consequence: such data persists, and re-importing the app's own CSV export throws Invalid month format from App.utils.normalizeMonthKey at index.html:1457–1466, aborting the entire restore. A blank input yields year 0 and negative years are accepted.
Task: Add min="1970", max="2100" and step="1" to the year input at index.html:320, and in updateCalendarYear clamp the parsed number into that range, falling back to the current calendar year when the value is blank or not finite, before calling setCalendarState.
Constraints: Do not change App.utils.iso, toMonthKey or normalizeMonthKey in this change. Do not change the three localStorage key names groupLessonPlannerData, groupLessonPlannerSettings and paymentTemplate, and do not change the persisted data shape; this prompt prevents new corruption and deliberately does not migrate existing corrupt values. Keep the prev and next month buttons able to cross a year boundary within the allowed range. No new dependencies.
Acceptance criteria: Typing 5 into the year input and committing leaves the calendar on a year within 1970 to 2100 and never produces a date key whose year part is shorter than four characters. Typing a blank value or a negative value does not set App.state.calYear to 0 or a negative number. Selecting any date in the calendar and pressing Done produces only date keys matching the pattern of four digits, hyphen, two digits, hyphen, two digits. Normal navigation across December to January still works.
Verification: Serve locally, open a group, open the schedule editor, type 5 then Enter in the year input and confirm the rendered year is clamped. Repeat with an empty value and with -3. Then select a date, press Done, read groupLessonPlannerData from localStorage and assert every entry in dates and every key of monthlyOverrides has a four-digit year.

PROMPT QW-4: Refresh the group list after a schedule save
Context: Repo lesson-planner, index.html. saveDateChanges at index.html:745–772 persists the new dates and monthly overrides and then calls App.render.monthlyOverrides(), App.render.groupInfo() and App.render.calendar(), but never App.render.groups(). The group card's lesson count is rendered from group.dates.length at index.html:1045 by render.groups.

Verified consequence: after selecting four dates and pressing Done, the card behind the modal still reads 0 planned lessons while the correct five dates are already persisted; a page reload shows the right number. The user is shown a wrong count and may redo work.
Task: Add a call to App.render.groups() inside saveDateChanges, after App.services.storage.save() at index.html:764, so the card count reflects the freshly saved dates.
Constraints: Do not change the three localStorage key names or the persisted data shape. Do not reorder or remove the existing render calls, and do not change resetEditingContext. Keep the change to a single added call; do not refactor saveDateChanges. Note that render.groups rebuilds the whole list with innerHTML and reattaches card click handlers, so no handler rewiring is needed. No new dependencies.
Acceptance criteria: Open a group with zero lessons, select four dates in the visible month, press Done, and the group card immediately reads 4 planned lessons without a reload. Reloading the page shows the same count. Pressing Cancel instead of Done leaves the card count unchanged. Deleting a group still empties the list correctly.
Verification: Serve locally, add a group, open it, open the schedule editor, click a weekday header to select four dates, press Done, then read the .group-card text content and assert it contains 4 planned lessons. Reload and assert the same text.

PROMPT QW-5: Fix pluralisation in the lesson and day count strings
Context: Repo lesson-planner, index.html. Two user-facing strings hardcode a plural noun. The month row at index.html:1089 renders (${lessonCount} lessons), producing (1 lessons). The calendar summary at index.html:1172 renders ${selectedInMonthCount} days selected in ..., producing 1 days selected in September. Both were observed in a running browser. These strings are also intended test anchors, so their exact wording should be settled before an automated suite asserts on them.
Task: Introduce a small helper in App.utils that returns a correctly pluralised noun for a count, and use it for the lessons noun at index.html:1089 and the days noun at index.html:1172 so a count of 1 renders lesson and day while all other counts render lessons and days.
Constraints: Change only the noun. Keep the rest of each string byte-identical, including the parentheses around the month row count, the word selected in, the literal Total: prefix and the em dash character in the calendar summary. Do not change the group card string {n} planned lessons at index.html:1049, which is out of scope for this prompt. Do not change the three localStorage key names or the persisted data shape. No new dependencies and no internationalisation library; English-only pluralisation is sufficient because all formatting is already pinned to en-US.
Acceptance criteria: A month with exactly one lesson renders (1 lesson) and a month with four renders (4 lessons). A selection of exactly one date renders 1 day selected in {Month} — Total: {amount} with the em dash preserved, and two or more dates render N days selected. A selection of zero dates still renders an empty calendar summary.
Verification: Serve locally, open a group, open the schedule editor, select exactly one date and read the text of #calendar-summary and of the .month-override-row month name; then select a second date and read both again. Assert singular and plural forms respectively, and assert the summary still contains the em dash character.

PROMPT QW-6: Add a UTF-8 BOM to the CSV export so Cyrillic names survive Excel on Windows
Context: Repo lesson-planner, index.html. saveToCsv at index.html:915-931 builds the CSV with App.services.csv.serialize and wraps it in new Blob([csvContent], { type: 'text/csv;charset=utf-8;' }) at index.html:921. Inspection of a real downloaded file confirmed the bytes begin with the characters of "Name" and carry no UTF-8 byte order mark. The app's target user has Cyrillic group names, and Excel on Windows misreads BOM-less UTF-8 CSV as a legacy code page, so those names appear as mojibake in the file that is the app's only backup.
Task: Prepend the single UTF-8 BOM character U+FEFF to the Blob contents in saveToCsv at

index.html:921 so the downloaded file starts with the bytes EF BB BF, leaving App.services.csv.serialize itself unchanged. No importer change is required: the header lookup at index.html:1289 lowercases a trimmed cell, and String.prototype.trim strips U+FEFF, so the first column still resolves to name. Confirm this with the round-trip check below rather than assuming it.
Constraints: Do not change the header row text, the CRLF row endings, the all-fields-quoted convention or the column order Name, Default Price, Currency, Month, Month Price, Dates. Do not change the download filename pattern at index.html:923-926. Do not change the three localStorage key names or the persisted data shape. No new dependencies. Out of scope: adding the template to the export, which is tracked separately as defect D16.
Acceptance criteria: A downloaded export begins with the three bytes EF BB BF. The rest of the file is byte-identical to the previous output for the same data. Importing that exact downloaded file restores the same groups, dates, per-month prices and currency, with no alert shown, which proves the required-column lookup still finds the name column despite the BOM. A group name containing Cyrillic characters round-trips unchanged through export and import.
Verification: Serve locally, create a group whose name contains Cyrillic characters, select a date, press Done, click Save CSV, then inspect the first bytes of the downloaded file with a hex dump and assert efbbbf. Load that same file back with Load CSV and assert the group name, dates and price match, with no alert dialog.